

Week-6 Practical

1. APIRouter

It is a FastAPI class used to group related API routes into separate modules.

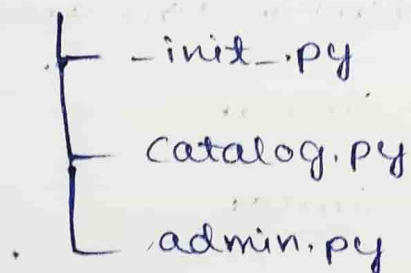
```
> from fastapi import APIRouter  
  
router = APIRouter()
```

It helps organize a large application into multiple files.

2. Modular Routing

It means dividing API endpoints into separate files according to their functionality.

Ex: routes/



- catalog.py → book viewing routes
- admin.py → book management routes

This improves code organization and maintainability.

3. include_router()

It adds routes from an APIRouter to the main FastAPI application.

> `app.include_router(catalog_router)`

`app.include_router(admin_router)`

The routes defined in separate files then become available through the main application.

4. Router Import and Alias

A router can be imported with an alias to make its purpose clear.

> `from routes.catalog import router as catalog_router`

Here:

`router` → imported router

`catalog_router` → local alias

Similarly:

`from routes.admin import router as admin_router`

5. `_init_.py`

It is a python file used to identify a directory as a python package.

An empty file is sufficient.

This allows modules inside routes to be imported conveniently.

6. Shared Mutable Data

A python list can be imported from one module and modified from another module.

```
from routes.catalog import books
```

The admin.py module can then modify the same books list:

```
books.append(new_book)
```

Changes made to the list are visible to other code using that same list during the application's runtime.

7. enumerate()

It provides both the index and value while iterating over the sequence.

```
> for index, book in enumerate(books):
```

```
...
```

Example:

```
index = 0, book = first book
```

```
index = 1, book = second book
```

It is useful when an element needs to be updated or deleted using its position.

8. `list.append()`

`append()` adds an item to the end of a list.

> `books.append(new_book)`

Example: `numbers = [1, 2]`

`numbers.append(3)`

Result:

`[1, 2, 3]`

9. `list.pop()`

`pop(index)` removes and returns an element at a specified index.

`deleted_book = books.pop(index)`

Useful to delete a particular book from list.

10. `max()` with default

`max()` returns the largest value in an iterable.

> `max(
 [b["id"] for b in books],
 default = 100
)`

`default = 100` provides a value when the list is empty.

The new ID is generated using:

`new_id = max(...) + 1`

11. List Comprehension

It provides a short way to create a list from another iterable.

To extract all books - `[b["id"] for b in books]`

Example: `books = [`
 `{ "id" : 101 },`
 `{ "id" : 102 }`
 `]`

produces : `[101, 102]`

12. Pydantic model-dump()

`model-dump()` converts a Pydantic model into a Python dictionary.

`book.model-dump()`

Example :

`Book (`

`title = "Python",`

`author = "John",`

`price = 450`

`)`

can be

converted
into

`{`

`"title": "Python",`

`"author": "John",`

`"price": 450`

`}`